
THE UNIVERSITY OF DODOMA

THE COLLEGE OF INFORMATICS AND VIRTUAL EDUCATION



DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING GROUP PROJECT REPORT ON

PWM-Based DC Motor Speed Control System

GROUP NO: 10

NAMES	REG NO	COURSE
NISAGURWE S AGOSTINO	T23-03-14465	BSc.CE
DAVID DANIEL DAVID	T23-03-15326	BSc.CE
DENNIS MARIWA	T23-03-19910	BSc.CE
EMMANUEL N FULUGE	T23-03-15286	BSc.CE

1. Introduction

Many computer engineering systems require precise control of the power delivered to an actuator. Examples include DC motor speed control, fan speed regulation, LED brightness adjustment, heater control, and small robotic drive systems. Simply switching a digital pin ON and OFF does not provide smooth control — it either delivers full power or none.

Pulse Width Modulation (PWM) solves this by allowing a microcontroller to control the average output power by varying the duty cycle of a digital waveform. By changing what proportion of each cycle the signal stays HIGH, the effective average voltage — and therefore the power — delivered to the load changes proportionally, without ever changing the supply voltage itself.

This project implements a complete PWM-based DC motor speed control system using the ATmega32 microcontroller. The system enables a user to increase or decrease motor speed through push buttons, while Timer0 hardware generates a PWM signal to drive the motor via a MOSFET switching stage. The project covers schematic and PCB design in KiCad, firmware development in MPLAB for VS Code, and functional simulation in SimulIDE.

2. Project Objectives

The objectives of this project are:

- To design and implement a PWM-based DC motor speed control system using the ATmega32 microcontroller.
 - To configure Timer0 in Fast PWM mode to generate a variable-duty-cycle signal on pin PB3 (OC0).
 - To allow the user to control motor speed through three push buttons: increase, decrease, and stop.
 - To indicate the current speed level visually using three LED indicators on Port B.
 - To protect the microcontroller using a MOSFET driver stage with a flyback diode.
 - To design the complete circuit schematic and PCB layout using KiCad, with ERC/DRC compliance.
 - To simulate the system in SimulIDE and verify PWM waveform generation and button response.
 - To document all design decisions, calculations, and test results in a GitHub repository.
-

3. System Overview

3.1 Block Diagram

The following block diagram illustrates the overall system architecture:

System Block Diagram

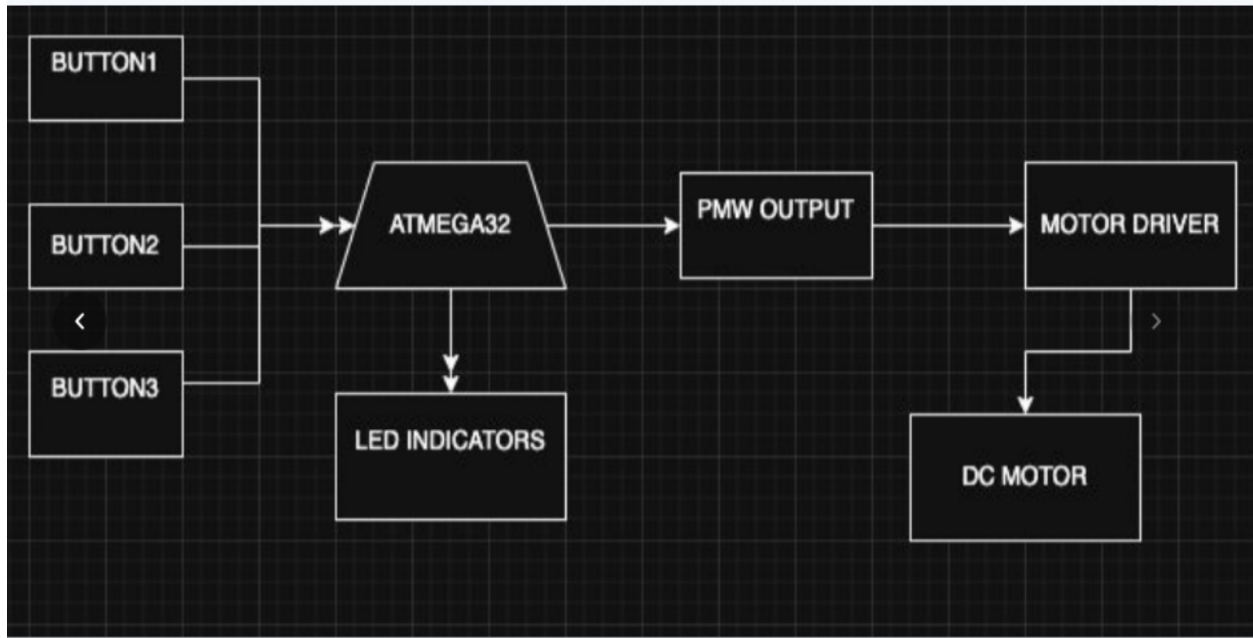


Figure: System Block Diagram

3.2 System Description

The system consists of five functional blocks:

Push Button Input Stage

Three push buttons connected to Port A (PA0, PA1, PA2) allow the user to increase speed, decrease speed, or stop the motor. Internal pull-ups are enabled in firmware, so buttons connect to GND when pressed (active-low logic). A fourth button on the RESET pin allows manual microcontroller reset.

ATmega32 Microcontroller

The ATmega32 serves as the central controller. It reads button states, updates the OCR0 register to set the PWM duty cycle, and drives the LED indicators. Timer0 operates in hardware Fast PWM mode and automatically produces the PWM output on PB3 (OC0) without requiring manual pin toggling in the main loop.

MOSFET Driver Stage

The ATmega32 cannot drive a DC motor directly — its pins can only source or sink a few milliamps. An N-channel MOSFET (Q1) acts as a low-side switch: when the PWM signal is HIGH, the MOSFET conducts and current flows through the motor; when LOW, the MOSFET cuts off. A 220 ohm gate resistor (R2) limits the current from PB3 during switching.

Flyback Diode Protection

A 1N4007 diode (D4) is placed across the motor terminals in reverse bias. When the MOSFET turns off, the motor's inductive load generates a reverse voltage spike (back-EMF). Without this diode, the spike could destroy the MOSFET and damage the microcontroller. The diode provides a safe freewheeling path for this energy.

LED Speed Indicators

Three LEDs connected to PB0, PB1, and PB2 through 330 ohm current-limiting resistors provide a visual indication of the selected speed level. The firmware updates these LEDs each time the duty cycle changes.

4. Hardware Design

4.1 Component List

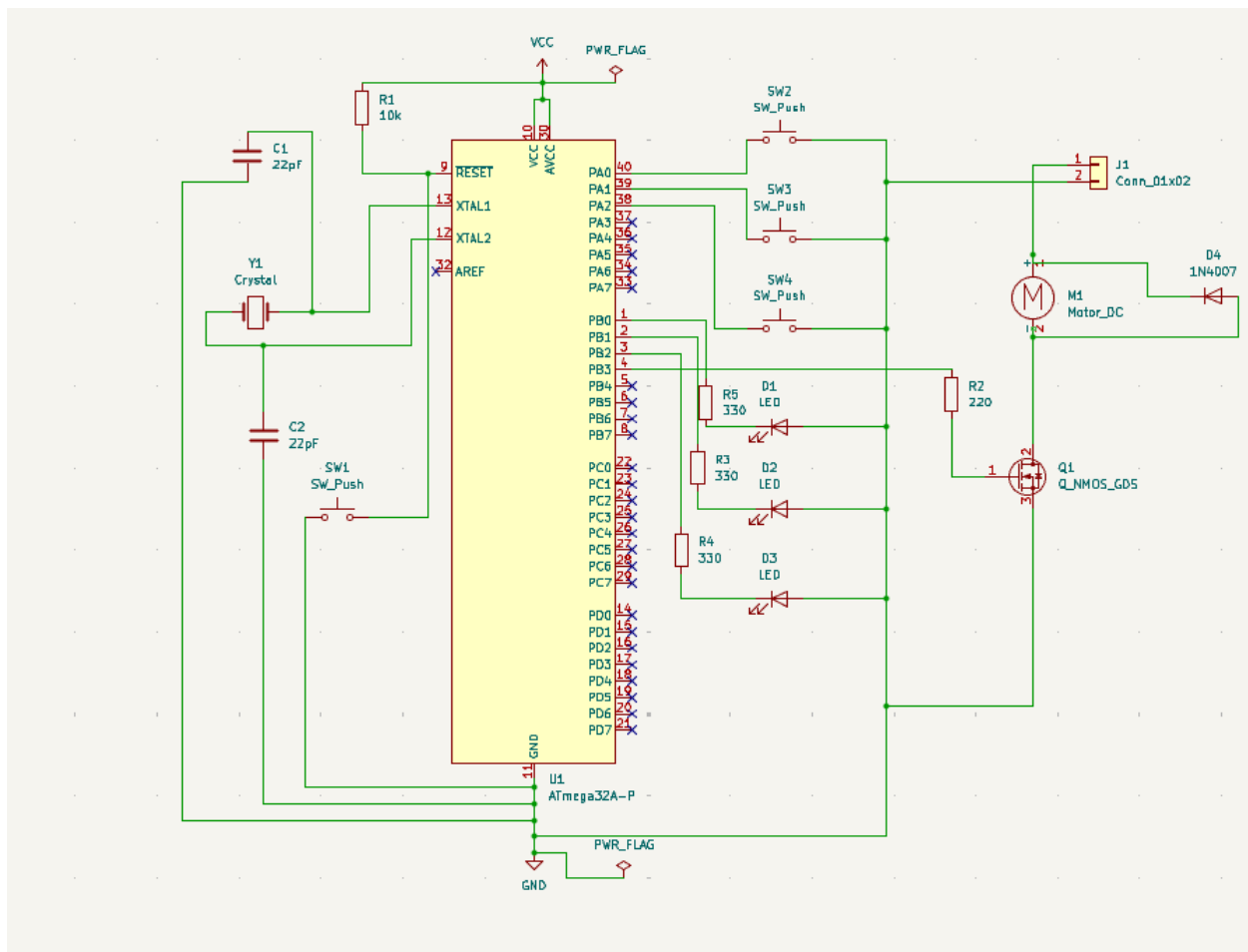
Component	Designator	Value / Part	Purpose
ATmega32 Microcontroller	U1	ATmega32A-P DIP-40	Main controller and PWM generator
DC Motor	M1	Small 5V DC motor	Controlled actuator
N-Channel MOSFET	Q1	Enhancement mode N-Ch	Low-side motor driver switch
Flyback Diode	D4	1N4007	Back-EMF protection
Gate Resistor	R2	220 ohm	Limits gate charge current from PB3
LED Low	D1	Standard LED	Speed level 1 indicator (25%)
LED Medium	D2	Standard LED	Speed level 2 indicator (50%)
LED High	D3	Standard LED	Speed level 3 indicator (75%/100%)
LED Resistors	R3-R5	330 ohm each	LED current limiting
Reset Pull-up	R1	10 kohm	Holds RESET pin HIGH normally
Crystal	Y1	8 MHz	External stable clock source
Load Capacitors	C1,C2	22 pF each	Crystal oscillator load capacitors
Increase Button	SW2	Push button	Speed increase input (PA0)
Decrease Button	SW3	Push button	Speed decrease input (PA1)
Stop Button	SW4	Push button	Motor stop input (PA2)
Reset Button	SW1	Push button	Manual microcontroller reset
Motor Connector	J1	2-pin connector	External motor power supply

4.2 Pin Mapping Table

ATmega32 Pin	Net Label	Connected Device	Direction
PB3 / OC0	PWM_MOTOR	220 ohm (R2) → MOSFET Gate (Q1)	Output — PWM
PB0	LED_LOW	LED D1 + 330 ohm (R3)	Output
PB1	LED_MED	LED D2 + 330 ohm (R4)	Output
PB2	LED_HIGH	LED D3 + 330 ohm (R5)	Output
PA0	BTN_INC	Increase push button SW2	Input (internal pull-up)
PA1	BTN_DEC	Decrease push button SW3	Input (internal pull-up)
PA2	BTN_STOP	Stop push button SW4	Input (internal pull-up)
RESET	—	SW1 + R1 (10 kohm to VCC)	Input
XTAL1	—	Crystal Y1 + C1 (22 pF to GND)	Clock
XTAL2	—	Crystal Y1 + C2 (22 pF to GND)	Clock
VCC / AVCC	VCC	+5V power supply	Power
GND	GND	Common ground (logic + motor)	Power

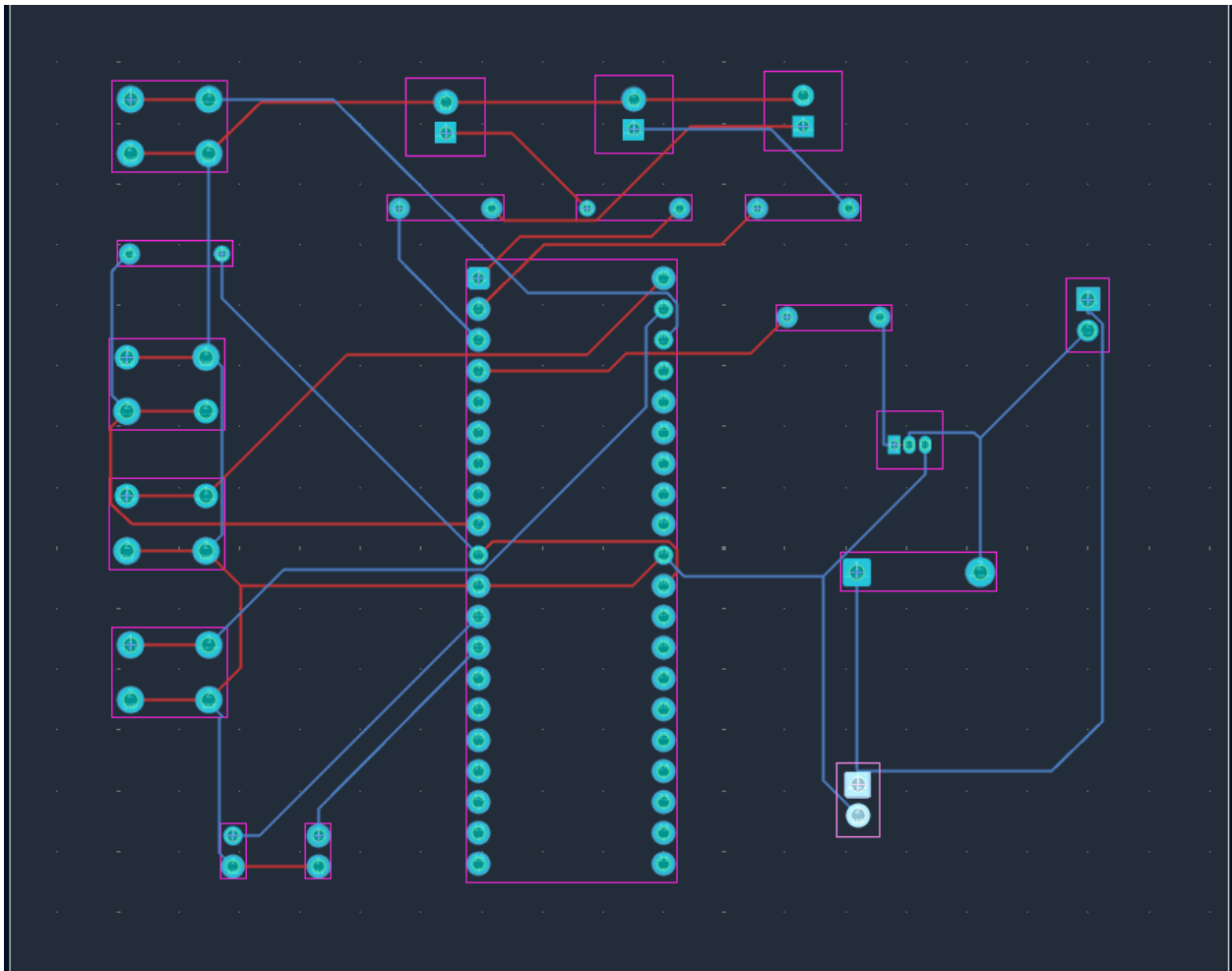
4.3 KiCad Schematic

The complete circuit schematic was designed in KiCad. Net labels (PWM_MOTOR, BTN_INC, BTN_DEC, BTN_STOP, LED_LOW, LED_MED, LED_HIGH) were applied to all key signal nets for clarity and to facilitate PCB routing.



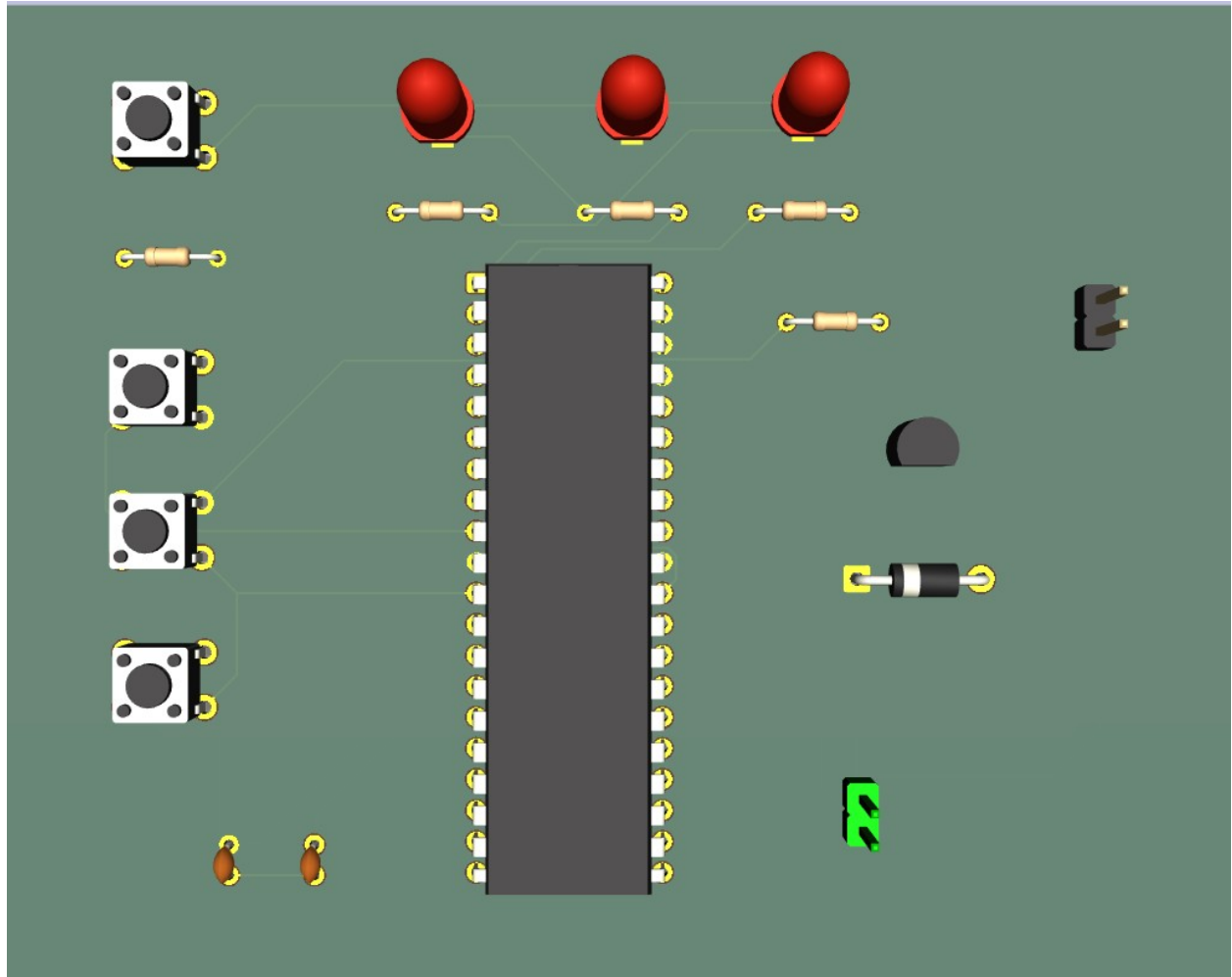
4.4 PCB Layout — 2D View

The PCB layout was designed in KiCad. The motor driver stage (Q1, R2, D4, J1) was placed away from the crystal oscillator (Y1, C1, C2) to minimise high-frequency switching noise coupling into the sensitive clock circuit. Wider copper traces were used for the motor current path. Design Rule Check (DRC) and Electrical Rule Check (ERC) were completed before Gerber file generation.



4.5 PCB Layout — 3D View

The 3D view was generated using KiCad's built-in 3D Viewer (View → 3D Viewer) to verify component placement and physical clearances before fabrication



5. PWM Configuration

5.1 Timer Selection and Mode

Timer/Counter0 (Timer0) of the ATmega32 was selected as the PWM source and configured in Fast PWM mode (8-bit). In this mode, the timer counts from 0 to 255 continuously and resets automatically. The PWM output is produced on pin PB3 (OC0) in non-inverting mode: the pin is driven HIGH at the start of each cycle (counter = 0) and driven LOW when the counter value reaches OCR0.

5.2 TCCR0 Register Configuration

Bit Field	Setting	Value	Meaning
WGM01:WGM00	Waveform Generation Mode	1:1	Fast PWM, 8-bit (TOP = 0xFF = 255)
COM01:COM00	Compare Output Mode	1:0	Clear OC0 on match, set at BOTTOM (non-inverting)
CS02:CS00	Clock Select	001	No prescaler — clock = CLK/1

The resulting firmware initialization code:

```
DDRB  |= (1 << PB3);           // OC0 pin as output
TCCR0 = (1 << WGM01)|(1 << WGM00)|(1 << COM01)|(1 << CS00); // Fast PWM,
non-inv, no prescaler
OCR0  = 0;                     // Start stopped
```

5.3 PWM Frequency Calculation

For 8-bit Fast PWM mode, the PWM output frequency is determined by:

$$f_PWM = f_clk / (N \times 256)$$

Where: $f_clk = 8,000,000$ Hz (external 8 MHz crystal oscillator), $N = 1$ (no prescaler), and 256 is the 8-bit timer resolution (counts 0 to 255 inclusive).

$$f_PWM = 8,000,000 / (1 \times 256) = 31,250 \text{ Hz (31.25 kHz)}$$

This frequency is fixed and does not change with duty cycle — only the proportion of time the signal remains HIGH within each 32-microsecond period varies with the OCR0 value.

Note: During SimulIDE simulation, the oscilloscope measured approximately 3.9 kHz rather than 31.25 kHz. This is because the simulator's internal clock configuration may differ from the target 8 MHz hardware. The PWM duty cycle behavior was verified to be correct across all five levels regardless of the simulated frequency reading.

6. Duty Cycle Calculations

6.1 Formula

The duty cycle is determined by the value of the OCR0 (Output Compare Register 0) relative to the 8-bit timer maximum of 255:

$$D = (\text{OCR0} / 255) \times 100\%$$

6.2 Speed Levels and OCR0 Values

Level	OCR0 Value	Duty Cycle	T_ON (at 31.25 kHz)	Motor Condition
0	0	0%	0 μ s	Stopped
1	64	25.1%	8 μ s	Low speed
2	128	50.2%	16 μ s	Medium speed
3	191	74.9%	24 μ s	High speed
4	255	100%	32 μ s	Full speed

6.3 Worked Calculations

Level 1 — Low Speed

$$D = (64 / 255) \times 100\% = 25.1\%$$

Level 2 — Medium Speed

$$D = (128 / 255) \times 100\% = 50.2\%$$

Level 3 — High Speed

$$D = (191 / 255) \times 100\% = 74.9\%$$

6.4 Button-to-OCR0 Mapping

Button	Pin	Action	Effect on OCR0	Result
Increase	PA0	Pressed	OCR0 += 64 (capped at 255)	Duty cycle increases by ~25%
Decrease	PA1	Pressed	OCR0 -= 64 (floored at 0)	Duty cycle decreases by ~25%
Stop	PA2	Pressed	OCR0 = 0	Motor stops immediately

7. Firmware Description

7.1 Repository Firmware Structure

File	Location	Description
main.c	firmware/src/	Main program — calls init functions and runs the main loop
pwm_motor.c	firmware/src/	Function definitions for PWM, GPIO, buttons, and LED control
pwm_motor.h	firmware/include/	Header file — function prototypes and includes
main.hex	firmware/hex/	Compiled output loaded into SimulIDE and the physical ATmega32
main.elf	firmware/build/	ELF debug binary (not required for GitHub submission)

7.2 Main Program Structure

The firmware follows the modular structure specified in the project brief:

```
#include "pwm_motor.h"

volatile uint8_t speed_level = SPEED_MIN;

int main(void) { initialize_gpio(); initialize_pwm();

    speed_level = SPEED_MIN;
    update_duty_cycle();
    update_speed_indicators();

    sei();

    while (1)
    {
        read_buttons();
        update_duty_cycle();
        update_speed_indicators();
    }

    return 0;

}
```

7.3 Key Functions (pwm_motor.c)

Function	Description
initialize_gpio()	Sets DDRB bits PB0-PB3 as outputs (LEDs + PWM pin). Sets DDRA PA0-PA2 as inputs. Enables internal pull-ups on PA0, PA1, PA2 via PORTA = 0x07.
initialize_pwm()	Sets TCCR0 = (WGM01 WGM00 COM01 CS00) for Fast PWM non-inverting, no prescaler. Sets OCR0 = 0 to start with motor stopped.
read_buttons()	Polls PINA for active-low button presses. Applies 20ms software debounce delay on each press. Calls appropriate OCR0 update function.
update_speed_indicators()	Clears PB0, PB1, PB2 then re-sets them based on OCR0 threshold: PB0 ON if OCR0>=64, PB1 ON if OCR0>=128, PB2 ON if OCR0>=191.

7.4 Firmware Screenshot

The following screenshot shows the firmware source code in MPLAB for VS Code:

Figure: MPLAB for VS Code — Firmware Source Code Screenshot

```

1  #include "pwm_motor.h"
2
3  extern volatile uint8_t speed_level;
4
5  static const uint8_t ocr0_table[SPEED_LEVELS] =
6  {
7      OCR0_LEVEL_0,
8      OCR0_LEVEL_1,
9      OCR0_LEVEL_2,
10     OCR0_LEVEL_3,
11     OCR0_LEVEL_4
12 };
13
14 void initialize_gpio(void)
15 {
16     BTN_DDR  &= ~((1 << BTN_INC_PIN) | (1 << BTN_DEC_PIN) | (1 << BTN_STOP_PIN));
17     BTN_PORT |= ((1 << BTN_INC_PIN) | (1 << BTN_DEC_PIN) | (1 << BTN_STOP_PIN));
18
19     LED_DDR  |= ((1 << LED_LOW_PIN) | (1 << LED_MED_PIN) | (1 << LED_HIGH_PIN));
20     LED_PORT &= ~((1 << LED_LOW_PIN) | (1 << LED_MED_PIN) | (1 << LED_HIGH_PIN));
21
22     PWM_DDR  |= (1 << PWM_PIN);
23 }
24
25 void initialize_pwm(void)
26 {
27     TCCR0 = (1 << WGM01) | (1 << WGM00) | (1 << COM01) |
28             (0 << CS02) | (1 << CS01) | (0 << CS00);
29
30     OCR0 = OCR0_LEVEL_0;
31 }
32
33 void set_pwm_duty_cycle(uint8_t level)
34 {
35     if (level > SPEED_MAX)
36     {
37         level = SPEED_MAX;
38     }
39     OCR0 = ocr0_table[level];
40 }
41
42 uint8_t debounce_button(uint8_t pin_mask)
43 {
44     if (!(BTN_PIN & pin_mask))
45     {
46         _delay_ms(DEBOUNCE_DELAY_MS);
47         if (!(BTN_PIN & pin_mask))
48         {
49             return 1;
50         }
51     }
52     return 0;
53 }
54
55 void read_buttons(void)
56 {
57     if (debounce_button(1 << BTN_STOP_PIN))
58     {

```

```

59     speed_level = SPEED_MIN;
60     while (!(BTN_PIN & (1 << BTN_STOP_PIN)));
61     _delay_ms(DEBOUNCE_DELAY_MS);
62     return;
63 }
64
65 if (debounce_button(1 << BTN_INC_PIN))
66 {
67     if (speed_level < SPEED_MAX)
68     {
69         speed_level++;
70     }
71     while (!(BTN_PIN & (1 << BTN_INC_PIN)));
72     _delay_ms(DEBOUNCE_DELAY_MS);
73     return;
74 }
75
76 if (debounce_button(1 << BTN_DEC_PIN))
77 {
78     if (speed_level > SPEED_MIN)
79     {
80         speed_level--;
81     }
82     while (!(BTN_PIN & (1 << BTN_DEC_PIN)));
83     _delay_ms(DEBOUNCE_DELAY_MS);
84     return;
85 }
86 }
87
88 void update_duty_cycle(void)
89 {
90     set_pwm_duty_cycle(speed_level);
91 }
92
93 void update_speed_indicators(void)
94 {
95     LED_PORT &= ~(1 << LED_LOW_PIN) | (1 << LED_MED_PIN) | (1 << LED_HIGH_PIN));
96
97     switch (speed_level)
98     {
99         case 0:
100             break;
101         case 1:
102             LED_PORT |= (1 << LED_LOW_PIN);
103             break;
104         case 2:
105             LED_PORT |= (1 << LED_LOW_PIN) | (1 << LED_MED_PIN);
106             break;
107         case 3:
108         case 4:
109             LED_PORT |= (1 << LED_LOW_PIN) | (1 << LED_MED_PIN) | (1 << LED_HIGH_PIN);
110             break;
111         default:
112             break;
113     }
114 }

```

8. SimulIDE Simulation

8.1 Simulation Setup

The circuit was built and tested in SimulIDE prior to PCB fabrication. The simulation circuit includes:

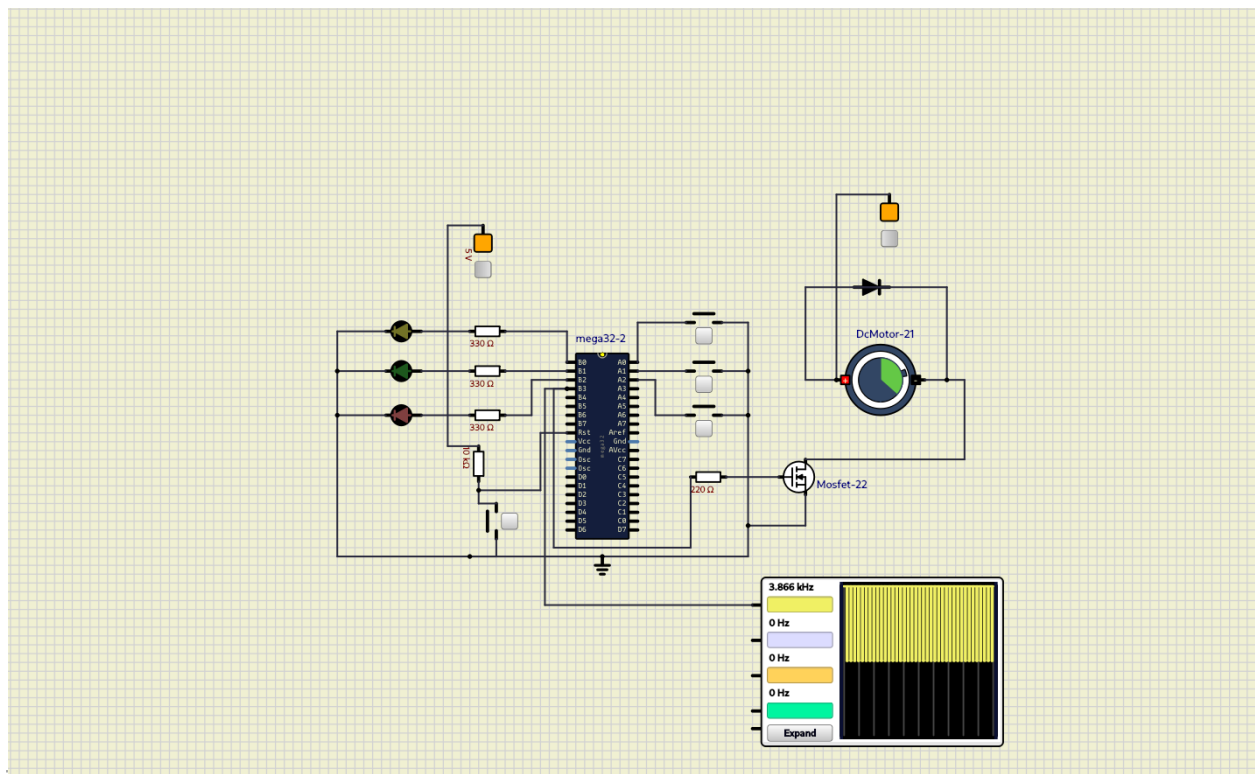
- ATmega32 (mega32-2 component) with the compiled main.hex firmware loaded
- Three push buttons connected to PA0, PA1, PA2 with active-low logic
- Three LEDs with 330 ohm resistors on PB0, PB1, PB2
- N-channel MOSFET (Enhancement mode, non-P-channel) with 220 ohm gate resistor from PB3
- DC motor (DcMotor component) connected through the MOSFET
- Flyback diode (1N4007) across the motor terminals
- 5V DC power source
- Oscilloscope probe connected to the PWM output node (PB3) to monitor duty cycle

8.2

Simulation Circuit Screenshot

8.3 PWM Waveform

The oscilloscope probe on PB3 confirmed that the PWM waveform duty cycle changed correctly each time the increase or decrease button was pressed. The screenshots below show the waveform at different duty cycle levels:



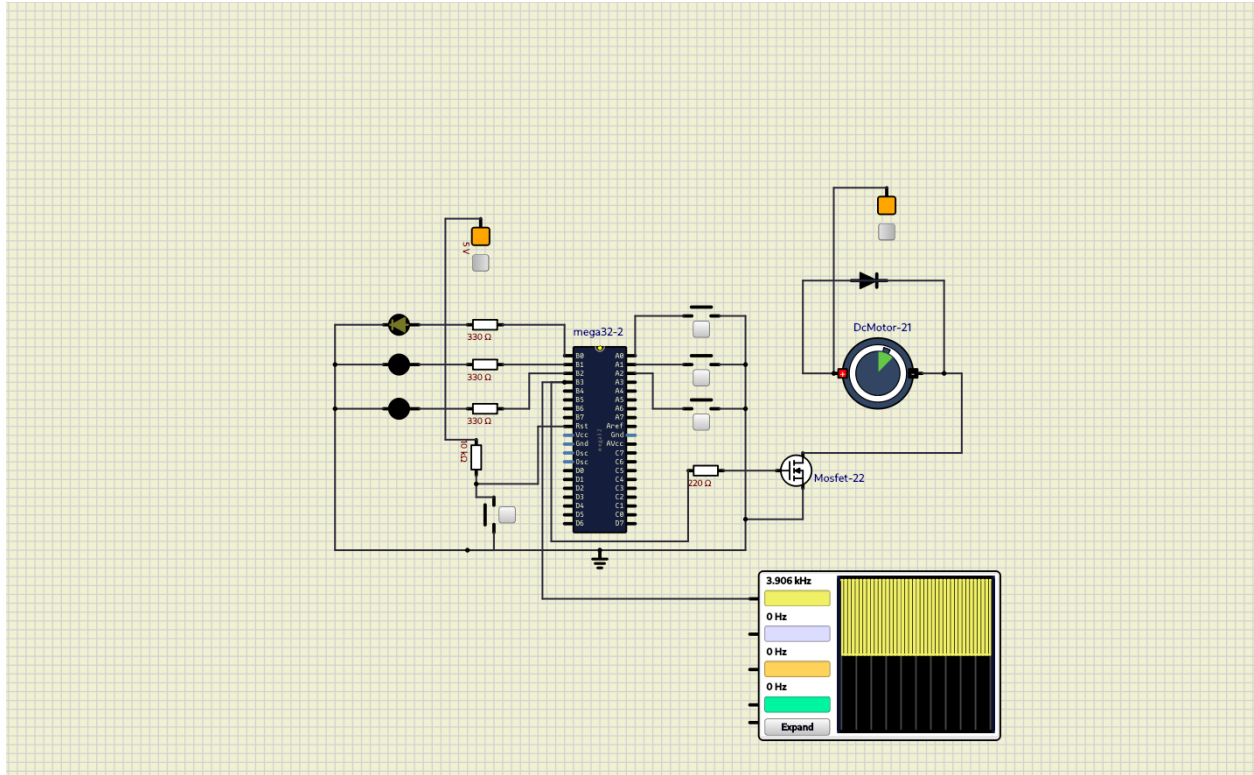


Figure: SimulIDE Oscilloscope — PWM Waveform at 50% Duty Cycle

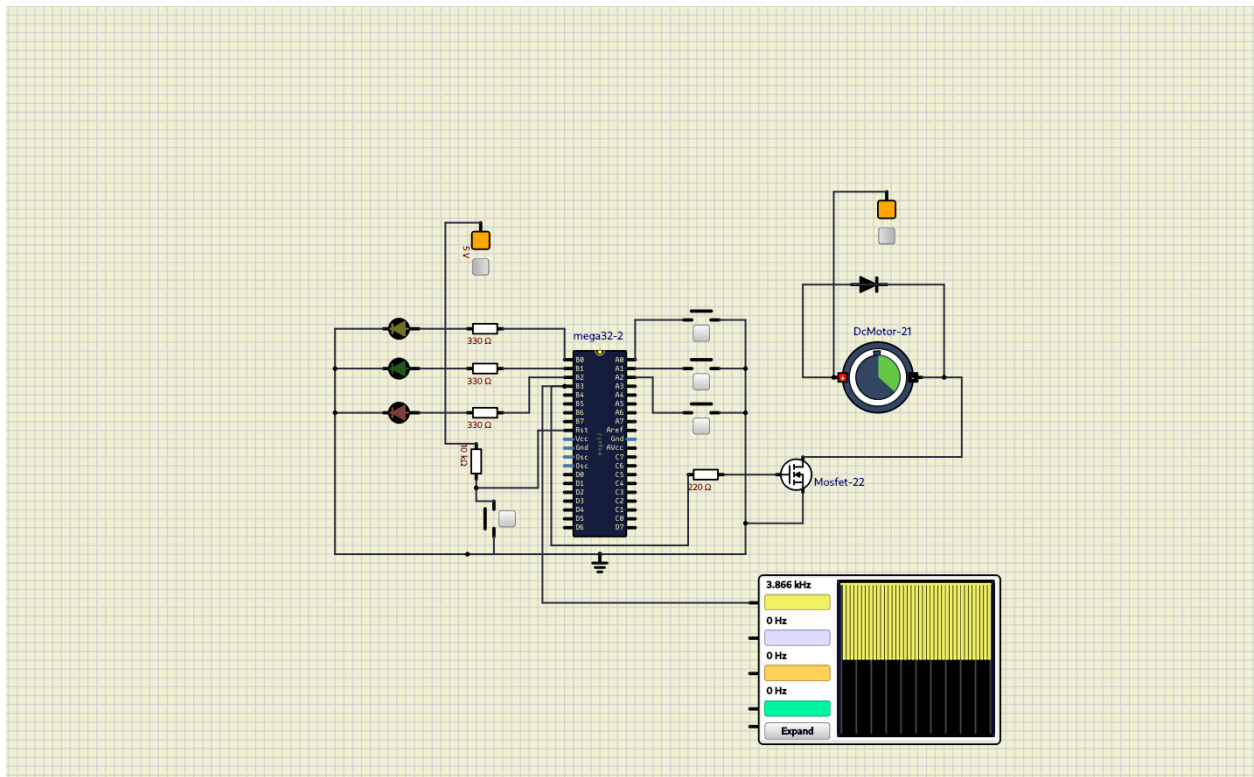


Figure: SimulIDE Oscilloscope — PWM Waveform at 25% or 75%

8.4 Test Results

Test	Action Taken	Expected Result	Observed Result	Status
System startup	Power ON simulation	Motor stopped, all LEDs OFF	Motor stopped, LEDs OFF	PASS
Increase × 1	Press BTN_INC once	OCR0=64, 25%, LED D1 ON	LED D1 lit, waveform changed	PASS
Increase × 2	Press BTN_INC again	OCR0=128, 50%, LED D2 ON	LED D2 lit, waveform changed	PASS
Increase × 3	Press BTN_INC again	OCR0=191, 75%, LED D3 ON	LED D3 lit, waveform changed	PASS
Full speed	Press BTN_INC again	OCR0=255, 100%, motor at max	Motor at full speed	PASS
Upper limit	Press BTN_INC at 100%	No change — capped at 255	No change observed	PASS
Decrease speed	Press BTN_DEC	OCR0 drops by 64, speed decreases	Speed decreased, LED updated	PASS
Stop function	Press BTN_STOP	OCR0=0, motor stops immediately	Motor stopped, all LEDs OFF	PASS
Lower limit	Press BTN_DEC at 0%	No change — floored at 0	No change observed	PASS
Reset	Press RESET button	MCU restarts, motor starts stopped	System reset correctly	PASS
PWM waveform	Observe oscilloscope at PB3	Duty cycle changes visible on scope	Waveform duty cycle confirmed correct	PASS

9. GitHub Repository

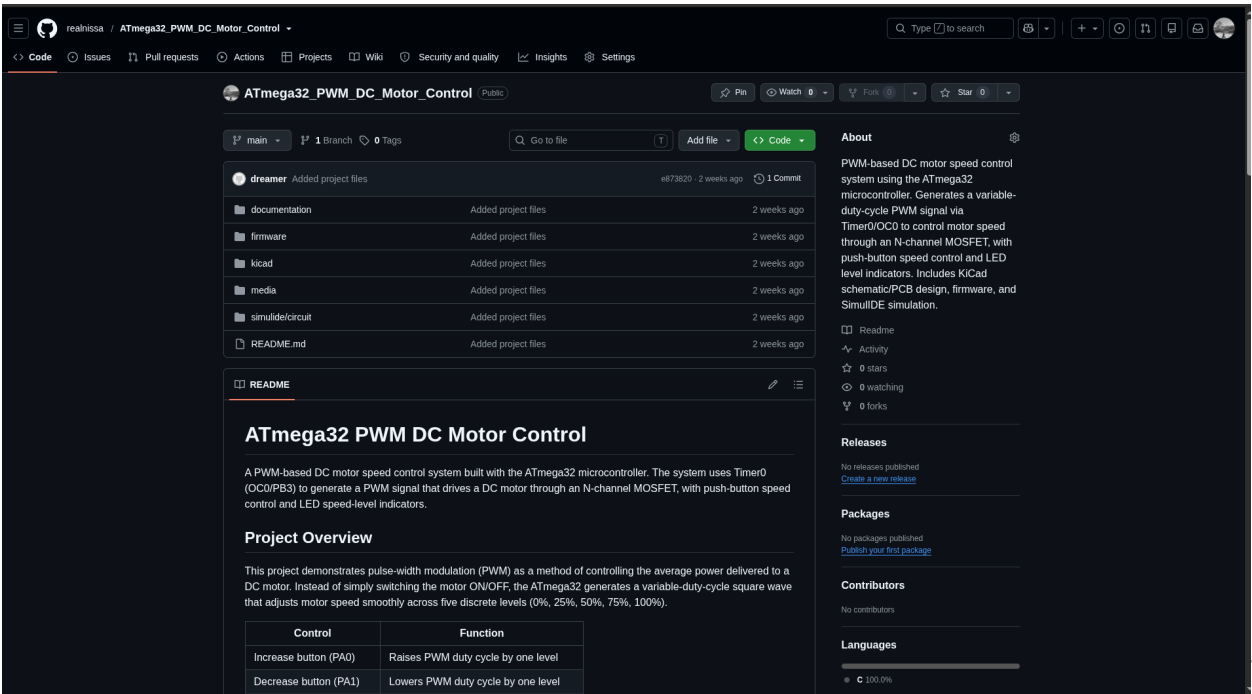
All project files are organised and submitted through the GitHub repository at:

https://github.com/realnissa/ATmega32_PWM_DC_Motor_Control

9.1 Repository Structure

Folder / File	Contents
firmware/src/	main.c, pwm_motor.c — C source code files
firmware/include/	pwm_motor.h — function prototypes and includes
firmware/hex/	main.hex — compiled firmware for programming and simulation
kicad/project/	.kicad_pro, .kicad_sch, .kicad_pcb — KiCad design files
kicad/gerber/	Gerber fabrication files (.gbr, .drl, -job.gbrjob, report.txt)
simulide/circuit/	pwm_motor_control.sim1 — SimulIDE circuit file
simulide/screenshots/	Simulation screenshots (PWM waveform, running circuit)
documentation/	pwm_configuration.md, duty_cycle_calculation.md, pin_mapping_table.md, test_results.md, demonstration_script.md
media/	demonstration_video_link.txt
README.md	Project overview, hardware summary, repository structure, and status table

9.2 Repository Screenshot



10. Conclusion

This project successfully designed and implemented a complete PWM-based DC motor speed control system using the ATmega32 microcontroller. The following objectives were achieved:

- Hardware PWM at 31.25 kHz was generated via Timer0 on pin PB3 (OC0) using Fast PWM mode — confirmed by oscilloscope in SimulIDE simulation.
- Five discrete speed levels (0%, 25%, 50%, 75%, 100%) were implemented and controlled accurately through OCR0 register manipulation.
- Three push buttons (increase, decrease, stop) provided reliable speed control with software debounce and safety limits preventing invalid OCR0 values.
- Three LED indicators on PB0, PB1, PB2 provided clear visual feedback of the current speed level.
- The MOSFET driver stage with 220 ohm gate resistor and 1N4007 flyback diode ensured the ATmega32 was protected from motor current and back-EMF.
- The complete circuit schematic and PCB layout were designed in KiCad, with DRC and ERC compliance verified.
- All 11 functional simulation tests passed in SimulIDE with the compiled hex firmware.
- All project files, documentation, and fabrication outputs are organised in the GitHub repository.

A possible future extension of this project is the implementation of closed-loop speed control using a rotary encoder. In this configuration, the ATmega32 would measure actual motor speed from encoder pulses using an input capture interrupt, and automatically adjust OCR0 to maintain a user-defined target speed regardless of load variations.

References

- Atmel Corporation. ATmega32/L Datasheet. Rev. 2503Q-AVR-02/2014.
- KiCad EDA Documentation. <https://docs.kicad.org>
- SimulIDE Documentation. <https://simulide.com/p/documentation/>
- CT 321 Project Brief — Project 10: PWM-Based DC Motor Speed Control System.
- Fairchild Semiconductor. 1N4007 General Purpose Rectifier Datasheet.
- MPLAB for VS Code Documentation. <https://www.microchip.com/mplab/mplab-for-vs-code>